Projekt 1: perceptron wielowarstwowy w zadaniu klasyfikacji na zbiorze MNIST

Wiktor Murawski, 333255

2 listopada 2026

Streszczenie

W pracy przedstawiono wyniki uczenia perceptronu wielowarstwowego (MLP) zaimplementowanego wyłącznie w NumPy na zbiorze MNIST. Zbadano wpływ architektury, funkcji aktywacji, parametrów SGD (learning rate, momentum) oraz rozmiaru mini-batcha. Najlepsza konfiguracja — `[784, 512, 256, 10]`, ReLU, SGD z lr = 0,05 i momentum = 0,9, batch 64 — osiągnęła accuracy na zbiorze testowym 98,61% ± 0,05 pp (średnia ± odchylenie standardowe z pięciu ziaren generatora).

1 Wstęp i cel

Celem było nauczenie sieci MLP rozpoznawania odręcznie zapisanych cyfr ze zbioru MNIST [1] przy użyciu własnej implementacji sieci (NumPy, propagacja wsteczna, SGD z momentum) z części pierwszej, bez rozszerzania modelu o nowe funkcjonalności.

Badano:

- architekturę (głębokość i szerokość warstw ukrytych),
- funkcję aktywacji w warstwach ukrytych,
- learning rate oraz momentum,
- rozmiar mini-batcha.

Uczenie prowadzono wyłącznie na zbiorze uczącym; zbiór testowy służył wyłącznie do końcowej oceny jakości.

2 Dane i metodologia

Tabela 1: Ustawienia wspólne dla eksperymentów.

Element	Wartość
Zbiór	MNIST (60 000 train / 10 000 test), piksele $\in [0, 1]$
Wejście / wyjście	784; 10 klas (softmax + cross-entropy)
Optymalizator bazowy	SGD, lr = 0,05, momentum = 0,9
Batch / epoki	64 / 30
Powtórzenia	seed 0–4; raport: średnia ± std accuracy (test)

Eksperymenty B–D budowano na zwycięzcy etapu poprzedniego (najpierw architektura, następnie aktywacja). Za bazową konfigurację przyjęto `[784, 256, 128, 10]`, ReLU, lr = 0,05, momentum = 0,9, batch 64.

3 Eksperymenty

3.1 Eksperyment A — architektura

Celem eksperymentu było sprawdzenie, jak na jakość klasyfikacji i czas uczenia wpływa pojemność sieci: liczba warstw ukrytych oraz liczba neuronów w warstwach. Porównano pięć wariantów — od płytkiej sieci z jedną warstwą ukrytą (`784-128-10`, `784-1024-10`), przez układy z dwiema warstwami ukrytymi o różnej szerokości (`784-256-128-10`, `784-512-256-10`), aż po głębszą sieć z trzema warstwami ukrytymi (`784-256-128-64-10`). Pozostałe hiperparametry utrzymano stałe: aktywacja ReLU, lr = 0,05, momentum = 0,9, batch 64, 30 epok.

Tabela 2: Wpływ architektury (ReLU, lr = 0,05, mom = 0,9, batch 64, 30 epok).

Architektura	Test accuracy [%]	Train accuracy [%]	Czas / przebieg [s]
784-128-10	$98{,}15 \pm 0{,}05$	100	∼41
784-256-128-10	$98{,}45 \pm 0{,}09$	100	∼120
784-256-128-64-10	$98{,}26 \pm 0{,}20$	99,95	∼136
784-512-256-10	$\mathbf{98{,}61 \pm 0{,}05}$	100	∼332
784-1024-10	$98{,}43 \pm 0{,}05$	100	∼506

Najlepszy wynik uzyskała sieć `[784, 512, 256, 10]`. Płytka sieć `784-128-10` pozostaje zaskakująco silna (≈ 98,15%) przy wielokrotnie krótszym czasie uczenia. Dodanie trzeciej warstwy ukrytej (`...-64-10`) nie poprawiło wyniku i zwiększyło rozrzut między seedami. Szeroka, płytka sieć z 1024 neuronami jest gorsza od `512-256` mimo dłuższego czasu — umiarkowana szerokość przy dwóch warstwach ukrytych okazała się korzystniejsza niż samo zwiększanie pojemności jednej warstwy. Train accuracy bliska 1,0 przy teście ≈ 98,6% wskazuje na lekki overfitting przy nadal dobrej generalizacji.

3.2 Eksperyment B — funkcja aktywacji

Na najlepszej architekturze z eksperymentu A (`784-512-256-10`) porównano trzy standardowe funkcje aktywacji w warstwach ukrytych: ReLU, tangens hiperboliczny oraz sigmoidę. Celem było zbadanie, która z nich lepiej wspiera uczenie przy tej samej szerokości sieci i identycznych ustawieniach SGD. Warstwa wyjściowa pozostała bez zmian (softmax).

Tabela 3: Porównanie funkcji aktywacji.

Aktywacja	Test accuracy [%]	Train accuracy [%]	Czas [s]
ReLU	$\mathbf{98{,}61 \pm 0{,}05}$	100	∼337
Tanh	$98{,}25 \pm 0{,}04$	100	∼378
Sigmoid	$98{,}08 \pm 0{,}06$	99,88	∼349

ReLU jest wyraźnie najlepsza (ok. +0,36 pp względem tanh i +0,53 pp względem sigmoid). Sigmoid wypada najsłabiej, co jest zgodne z problemem nasycenia i słabszych gradientów. Tanh jest stabilna (najmniejszy rozrzut), lecz systematycznie poniżej ReLU. Różnice są niewielkie, ale powtarzalne przy pięciu ziarnach.

3.3 Eksperyment C — learning rate i momentum

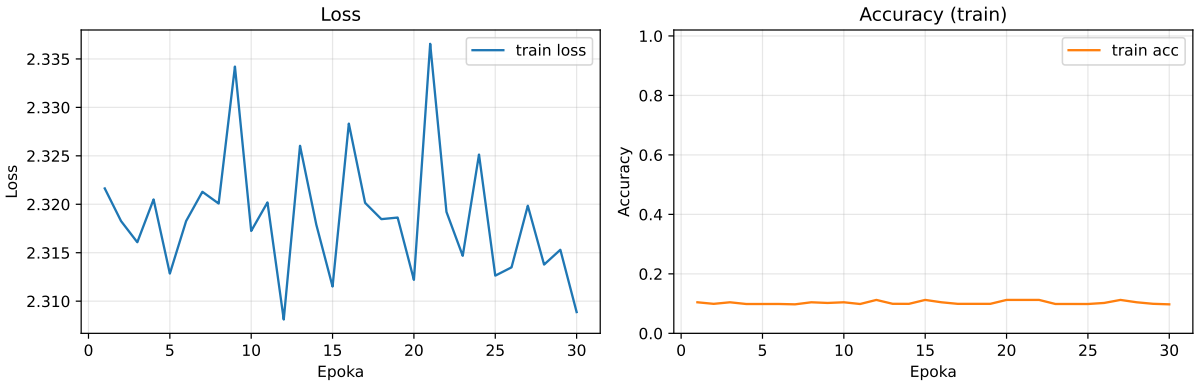
Przy ustalonej architekturze `784-512-256-10` i aktywacji ReLU zbadano wrażliwość uczenia na parametry optymalizatora SGD. Porównano trzy wartości learning rate (0,01, 0,05, 0,10) przy momentum 0,9, a także wariant bez momentum oraz z bardzo wysokim momentum 0,99 przy

lr = 0,05. Eksperyment miał pokazać zarówno obszar stabilnej zbieżności, jak i przypadki, w których dynamika aktualizacji wag się rozjeżdża.

Tabela 4: Wpływ learning rate i momentum.

Konfiguracja	Test accuracy [%]	Train accuracy [%]	Czas [s]
lr = 0,01, mom = 0,9	98,25 ± 0,05	100	∼346
lr = 0,05, mom = 0,9	**98,61 ± 0,05**	100	∼312
lr = 0,10, mom = 0,9	97,73 ± 0,20	99,5	∼302
lr = 0,05, mom = 0,0	98,20 ± 0,04	100	∼302
lr = 0,05, mom = 0,99	10,32 ± 0,55	10,2	∼280

Optimum to lr = 0,05 i momentum = 0,9. Zbyt duży krok (0,1) pogarsza wynik i zwiększa wariancję; zbyt mały (0,01) jest stabilny, lecz w 30 epokach ustępuje 0,05. Momentum 0,9 daje wyraźny zysk względem braku momentum (≈ +0,4 pp). Momentum 0,99 prowadzi do rozbieżności (accuracy rzędu losowego 10%). Ilustrację rozbieżności przedstawia rys. 1.



Rysunek 1: Przebieg uczenia dla lr = 0,05 i momentum = 0,99 (seed 0). Accuracy pozostaje na poziomie losowym (≈ 10%), a funkcja straty nie wykazuje stabilnej zbieżności.

3.4 Eksperyment D — rozmiar mini-batcha

Ostatni eksperyment dotyczył wpływu rozmiaru mini-batcha na accuracy i czas uczenia. Przy najlepszej dotychczas konfiguracji (architektura `784-512-256-10`, ReLU, lr = 0,05, momentum = 0,9) przetestowano batche 32, 64, 128 i 256. Mniejszy batch oznacza częstsze, bardziej zaszumione aktualizacje wag; większy — rzadsze, ale stabilniejsze kroki i zwykle krótszy czas jednej epoki.

Tabela 5: Wpływ batch size (najlepsza dotychczas konfiguracja).

Batch	Test accuracy [%]	Train accuracy [%]	Czas [s]
32	97,95 ± 0,26	99,68	∼471
64	**98,61 ± 0,05**	100	∼304
128	98,40 ± 0,05	100	∼231
256	98,31 ± 0,09	100	∼200

Batch 64 daje najlepszą accuracy. Większe batche (128, 256) skracają czas kosztem lekkiego spadku generalizacji. Batch 32 jest najwolniejszy, najgorszy wynikowo i najbardziej rozproszony

między seedami — przy lr = 0,05 szum gradientu okazuje się zbyt duży.

4 Podsumowanie

Tabela 6: Najlepsza znaleziona konfiguracja.

Parametr	Wartość
Architektura	`[784, 512, 256, 10]`
Aktywacja ukryta	ReLU
Wyjście / strata	softmax + cross-entropy
Optymalizator	SGD, lr = 0,05, momentum = 0,9
Batch size	64
Epoki	30
Wynik (test)	$\mathbf{98{,}61}\% \pm \mathbf{0{,}05}$ pp

Wnioski

Przeprowadzone eksperymenty pokazują, że nawet przy stosunkowo prostej implementacji MLP opartej wyłącznie o NumPy i SGD wybór architektury oraz hiperparametrów optymalizatora ma mierzalny i powtarzalny wpływ na jakość klasyfikacji. Różnice rzędu 0,3–0,8 punktu procentowego accuracy na zbiorze testowym utrzymywały się stabilnie przy pięciu różnych ziarnach generatora, co pozwala traktować je jako efekt konfiguracji, a nie szumu inicjalizacji czy kolejności mini-batchy.

Zwiększanie pojemności sieci nie przekłada się automatycznie na lepszy wynik. Szeroka, płytka architektura z 1024 neuronami w jednej warstwie ukrytej ustąpiła dwuwarstwowej sieci `512-256` mimo wyraźnie dłuższego czasu uczenia. Podobnie dodanie trzeciej warstwy ukrytej nie poprawiło generalizacji. W praktyce umiarkowana szerokość przy dwóch warstwach ukrytych okazała się korzystniejszym kompromisem między dokładnością a kosztem obliczeniowym.

Wśród funkcji aktywacji najskuteczniejsza była ReLU, przed tangensem hiperbolicznym i sigmoidą. Przewaga ReLU jest zgodna z literaturą: funkcja ta ogranicza nasycenie gradientu w porównaniu z sigmoidą i tangensem hiperbolicznym [2]. Sigmoida wypada najsłabiej, co można wiązać z klasycznym problemem zanikających gradientów w warstwach ukrytych.

Istotny okazał się też dobór parametrów SGD. Momentum na poziomie 0,9 systematycznie poprawiało wynik względem wariantu bez momentum, natomiast wartość 0,99 prowadziła do całkowitej utraty stabilności uczenia i accuracy bliskiej poziomowi losowemu. Learning rate 0,05 wypadł lepiej niż zarówno zbyt ostrożny krok 0,01, jak i zbyt agresywny 0,1. W połączeniu z mini-batchem o rozmiarze 64 konfiguracja ta dała najlepszy zaobserwowany kompromis między jakością a czasem uczenia; mniejsze batche zwiększały szum i czas, większe skracały obliczenia kosztem lekkiego spadku generalizacji.

Na koniec warto zwrócić uwagę na lukę między accuracy na zbiorze uczącym (bliską 100%) a wynikiem testowym (ok. 98,6%). Świadczy ona o overfittingu. Bez dodatkowej regularyzacji — celowo pominiętej w ramach ograniczeń zadania — trudno oczekiwać istotnie wyższych wyników na teście przy samym dłuższym trenowaniu lub dalszym powiększaniu sieci.

Ograniczenia

Brak dropoutu, weight decay, batch normalization, adaptacyjnych optymalizatorów oraz augmentacji. Dobór hiperparametrów w łańcuchu A→D opierał się o zbiór testowy, co jest metodycznie optymistyczne — w pełniejszym protokole należałoby wydzielić zbiór walidacyjny. Implementacja w NumPy jest czasowo kosztowna przy szerokich warstwach.

Możliwe rozszerzenia

Early stopping na walidacji, harmonogram learning rate, regularyzacja L2, porównanie z optymalizatorem Adam, augmentacja obrazów.

Literatura

[1] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[2] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning (ICML)*, pages 807–814, 2010.